

LAB 07: Burair Haidar, Syed Sualeh

TASK 1:

```
void UART_Printf(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    vsnprintf(uart_buffer, sizeof(uart_buffer), format, args);
    va_end(args);

    HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer), HAL_MAX_DELAY);
}

/**
 * @brief Send string over UART
 */
void UART_SendString(char *str)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)str, strlen(str), HAL_MAX_DELAY);
}

// ===== ADC FUNCTIONS =====

/**
 * @brief Read ADC value from channel 1 (PA1)
 * @return 12-bit ADC value (0-4095)
 */
uint32_t Read_ADC(void)
{
    uint32_t adc_value = 0;

    // Start ADC conversion
    HAL_ADC_Start(&hadc1);

    // Wait for conversion to complete with timeout
    if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK)
    {
        adc_value = HAL_ADC_GetValue(&hadc1);
    }

    // Stop ADC
    HAL_ADC_Stop(&hadc1);

    return adc_value;
}

/**
 * @brief Read averaged ADC value (multiple samples)
 * @return Averaged 12-bit ADC value
 */
uint32_t Read_ADC_Averaged(void)
{
    uint32_t sum = 0;

    // Take multiple samples
    for (int i = 0; i < ADC_BUFFER_SIZE; i++)
    {
```

```

    sum += Read_ADC();
    HAL_Delay(1); // Small delay between samples
}

return sum / ADC_BUFFER_SIZE;
}

/**
 * @brief Print ADC value with voltage (using integer math to avoid float issues)
 */
void Print_ADC_Value(uint32_t adc_value)
{
    // Calculate voltage in millivolts: (adc_value * 3300) / 4095
    uint32_t voltage_mv = (adc_value * ADC_REF_VOLTAGE) / ADC_MAX_VALUE;

    // Split into volts and millivolts for display
    uint32_t volts = voltage_mv / 1000;
    uint32_t millivolts = voltage_mv % 1000;

    UART_Printf("ADC: %4lu | Voltage: %lu.%03lu V\r\n",
        adc_value, volts, millivolts);
}

/* USER CODE BEGIN 2 */

// Calibrate ADC (optional but recommended)
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);

// Print header
UART_Printf("\033[2J\033[H"); // Clear screen
UART_Printf("=====\r\n");
UART_Printf(" LAB 7 TASK 1: ADC SENSOR READING\r\n");
UART_Printf("=====\r\n\r\n");
UART_Printf("Potentiometer on PA1 (ADC1_IN1)\r\n");
UART_Printf("ADC: 12-bit (0-4095) | Vref: 3.3V\r\n\r\n");
UART_Printf("Reading every 500ms...\r\n\r\n");

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // Read ADC value (averaged for stability)
    uint32_t adc_value = Read_ADC_Averaged();

    // Print the value with voltage
    Print_ADC_Value(adc_value);

    // Wait before next reading
    HAL_Delay(50);
}
/* USER CODE END 3 */
}

void SystemClock_Config(void)

```

```

{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};

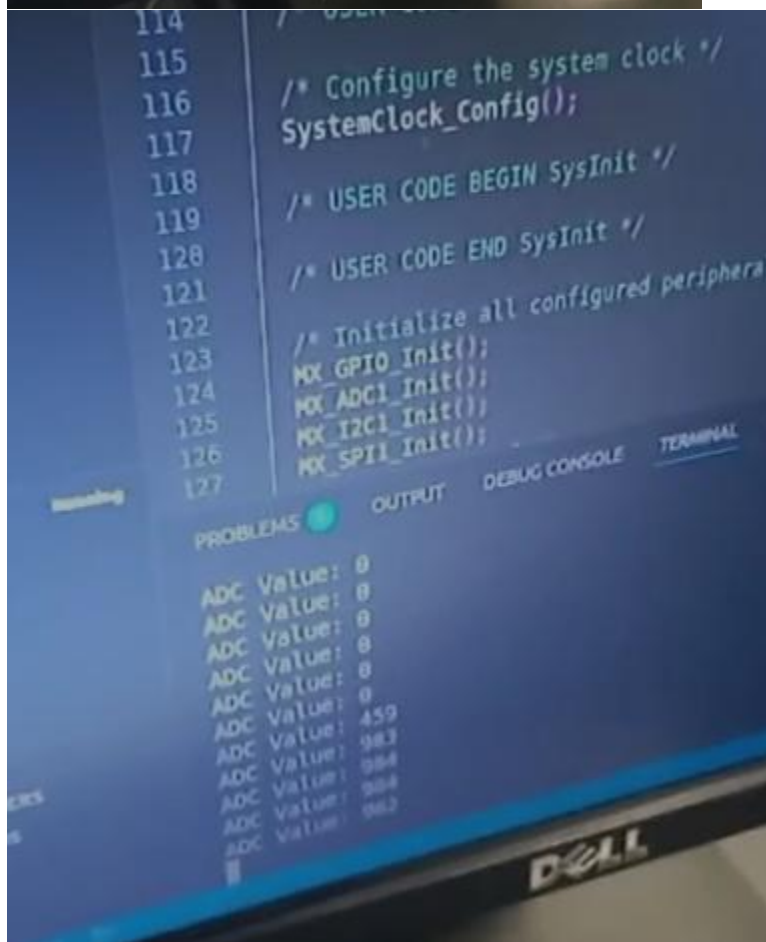
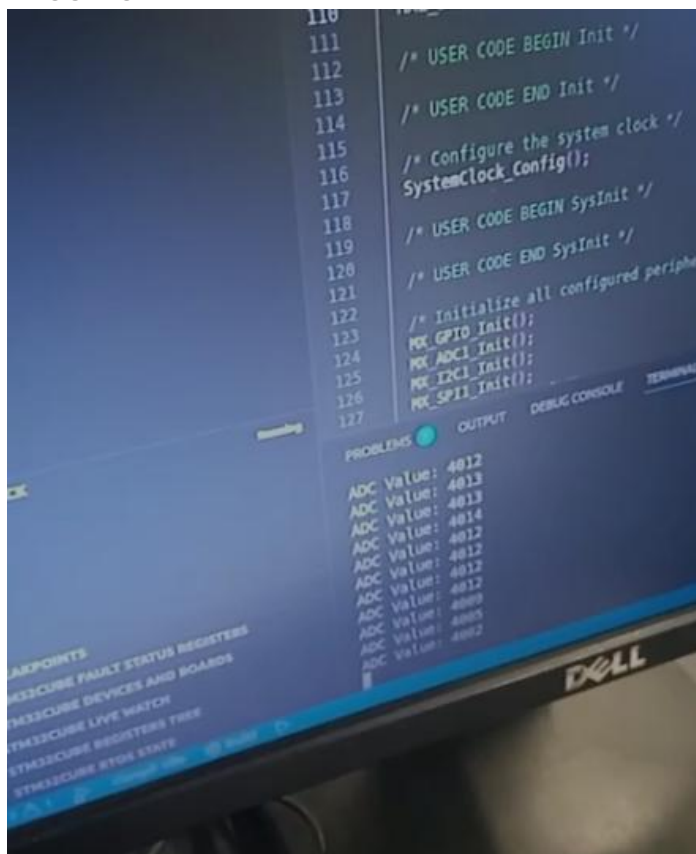
    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI|RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_BYPASS;
    RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL6;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSClkSource = RCC_SYSCCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
    {
        Error_Handler();
    }
    PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_USB|RCC_PERIPHCLK_I2C1
                                |RCC_PERIPHCLK_ADC12|RCC_PERIPHCLK_USART2;
    PeriphClkInit.Adc12ClockSelection = RCC_ADC12PLLCLK_DIV1;
    PeriphClkInit.I2c1ClockSelection = RCC_I2C1CLKSOURCE_HSI;
    PeriphClkInit.Usart2ClockSelection = RCC_USART2CLKSOURCE_PCLK1;
    PeriphClkInit.USBCLKSource = RCC_USBCLKSOURCE_PLL;
    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
    {
        Error_Handler();
    }
}

```

RESULTS:



TASK 2:

```
ADC_HandleTypeDef hadc1;
I2C_HandleTypeDef hi2c1;
SPI_HandleTypeDef hspi1;
UART_HandleTypeDef huart2;
PCD_HandleTypeDef hpcd_USB_FS;
/* USER CODE BEGIN PV */
// UART buffer for printf
static char uart_buffer[256];
// ADC sample buffers
static volatile uint16_t adc_raw_buffer[SAMPLE_COUNT];
static volatile float32_t adc_float_buffer[SAMPLE_COUNT];
static volatile float32_t filtered_buffer[SAMPLE_COUNT];
// Moving average filter buffer
static float32_t filter_buffer[FILTER_LEN];
static uint8_t filter_index = 0;
// Sampling control
static volatile uint16_t sample_index = 0;
static volatile uint8_t sampling_complete = 0;
static volatile uint32_t sampling_start_time = 0;
void UART_Printf(const char *format, ...)
{
    va_list args;
    va_start(args, format);
    vsnprintf(uart_buffer, sizeof(uart_buffer), format, args);
    va_end(args);

    HAL_UART_Transmit(&huart2, (uint8_t*)uart_buffer, strlen(uart_buffer), HAL_MAX_DELAY);
}

/**
 * @brief Send string over UART
 */
void UART_SendString(char *str)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)str, strlen(str), HAL_MAX_DELAY);
}

/**
 * @brief Send raw and filtered data to Python for plotting
 * Format: raw_value,filtered_value\n
 */
void Send_Data_to_Python(void)
{
    for (int i = 0; i < SAMPLE_COUNT; i++)
    {
        // Convert ADC raw value to voltage (mV) for better plotting
        uint32_t raw_mv = (adc_raw_buffer[i] * ADC_REF_VOLTAGE) / ADC_MAX_VALUE;
        uint32_t filtered_mv = (uint32_t)(filtered_buffer[i] * 1000); // Convert V to mV

        UART_Printf("%lu,%lu\r\n", raw_mv, filtered_mv);
    }
}
```

```
// ===== ADC FUNCTIONS =====
```

```
/**
```

```
 * @brief Initialize ADC
```

```
 */
```

```
void ADC_Init(void)
```

```
{
```

```
    HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
```

```
}
```

```
/**
```

```
 * @brief Start continuous ADC conversion (interrupt mode)
```

```
 */
```

```
void ADC_Start_Continuous(void)
```

```
{
```

```
    sample_index = 0;
```

```
    sampling_complete = 0;
```

```
    sampling_start_time = HAL_GetTick();
```

```
    HAL_ADC_Start_IT(&hadc1);
```

```
}
```

```
/**
```

```
 * @brief Stop continuous ADC conversion
```

```
 */
```

```
void ADC_Stop_Continuous(void)
```

```
{
```

```
    HAL_ADC_Stop_IT(&hadc1);
```

```
}
```

```
/**
```

```
 * @brief ADC conversion complete callback
```

```
 */
```

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
```

```
{
```

```
    if (hadc->Instance == ADC1)
```

```
    {
```

```
        // Store raw ADC value
```

```
        adc_raw_buffer[sample_index] = HAL_ADC_GetValue(hadc);
```

```
        // Convert to float for filtering (0.0 - 3.3V)
```

```
        adc_float_buffer[sample_index] = (adc_raw_buffer[sample_index] * 3.3f) / 4095.0f;
```

```
        sample_index++;
```

```
        // Check if we've collected enough samples
```

```
        if (sample_index >= SAMPLE_COUNT)
```

```
        {
```

```
            sampling_complete = 1;
```

```
            HAL_ADC_Stop_IT(hadc);
```

```
        }
```

```
    else
```

```

    {
        // Start next conversion
        HAL_ADC_Start_IT(hadc);
    }
}

// ===== FILTER FUNCTIONS =====

/**
 * @brief Initialize moving average filter
 */
void Moving_Average_Init(void)
{
    filter_index = 0;

    for (int i = 0; i < FILTER_LEN; i++)
    {
        filter_buffer[i] = 0.0f;
    }
}

float32_t Moving_Average_Filter(float32_t new_sample)
{
    float32_t sum = 0.0f;

    // Add new sample to buffer
    filter_buffer[filter_index] = new_sample;
    filter_index = (filter_index + 1) % FILTER_LEN;

    // Calculate average of all samples in buffer
    for (int i = 0; i < FILTER_LEN; i++)
    {
        sum += filter_buffer[i];
    }

    return sum / FILTER_LEN;
}

void Apply_Filter_to_Buffer(void)
{
    // Reset filter
    Moving_Average_Init();

    // Apply filter to each sample
    for (int i = 0; i < SAMPLE_COUNT; i++)
    {
        filtered_buffer[i] = Moving_Average_Filter(adc_float_buffer[i]);
    }
}

while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

```

```

// Start ADC sampling in interrupt mode
ADC_Start_Continuous();

// Wait for sampling to complete
while (!sampling_complete)
{
    HAL_Delay(1);
}

// Calculate actual sampling rate
uint32_t sampling_time = HAL_GetTick() - sampling_start_time;
float sampling_rate = (SAMPLE_COUNT * 1000.0f) / sampling_time;

UART_Printf("\r\nCollected %d samples in %lu ms\r\n",
    SAMPLE_COUNT, sampling_time);
UART_Printf("Sampling rate: %.2f Hz\r\n\r\n", sampling_rate);

// Apply filter to the buffer
Apply_Filter_to_Buffer();

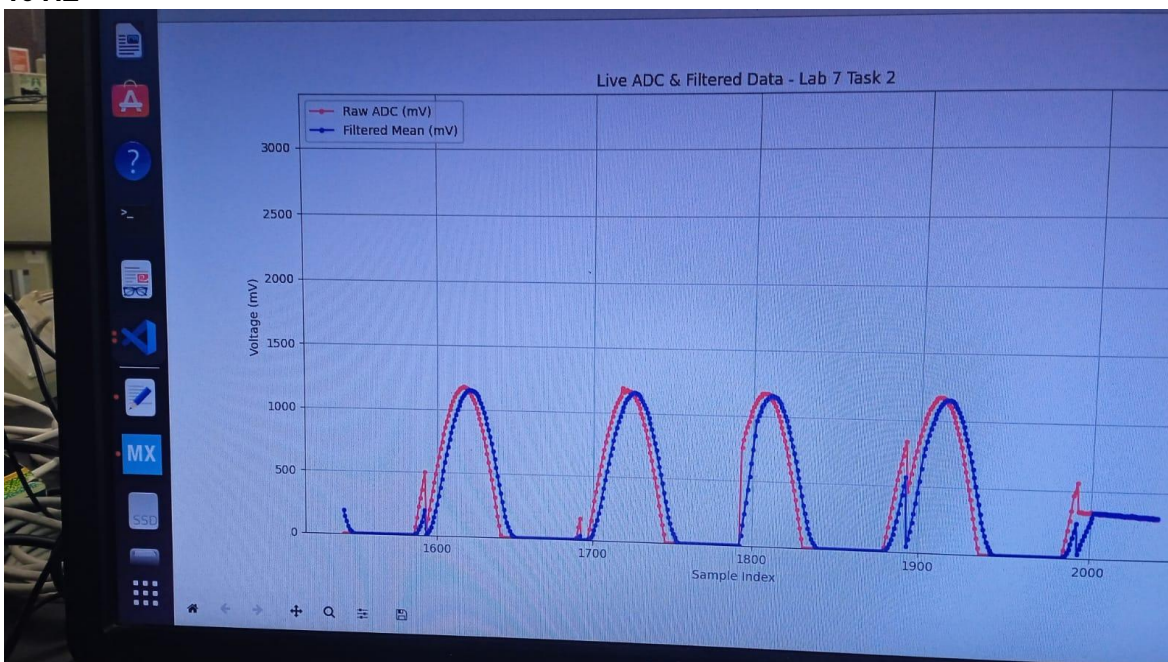
// Send data to Python for plotting
UART_Printf("=== DATA START ===\r\n");
Send_Data_to_Python();
UART_Printf("=== DATA END ===\r\n\r\n");

// Wait before next acquisition
UART_Printf("Waiting 2 seconds before next acquisition...\r\n\r\n");
HAL_Delay(2000);
}
/* USER CODE END 3 */
}

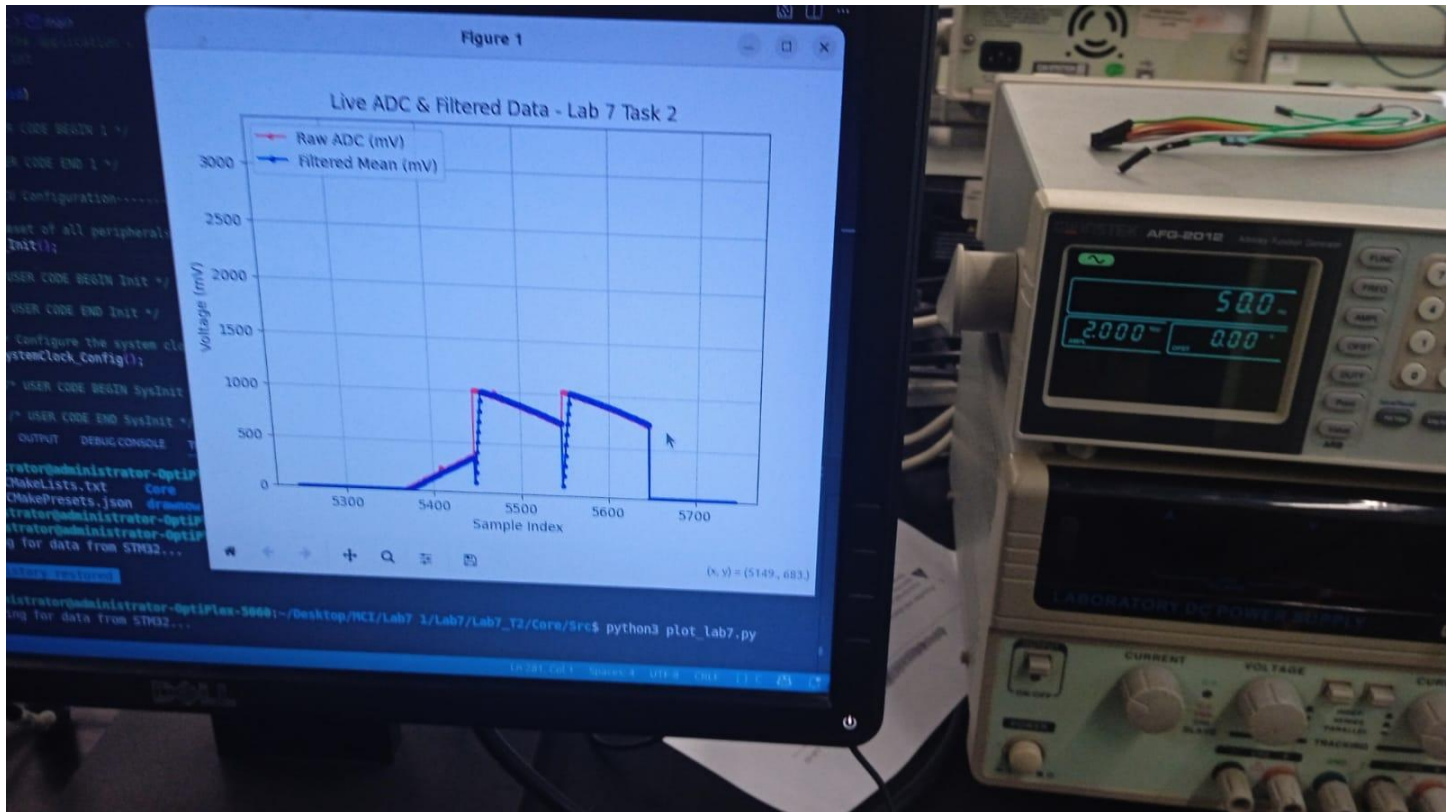
```

RESULT

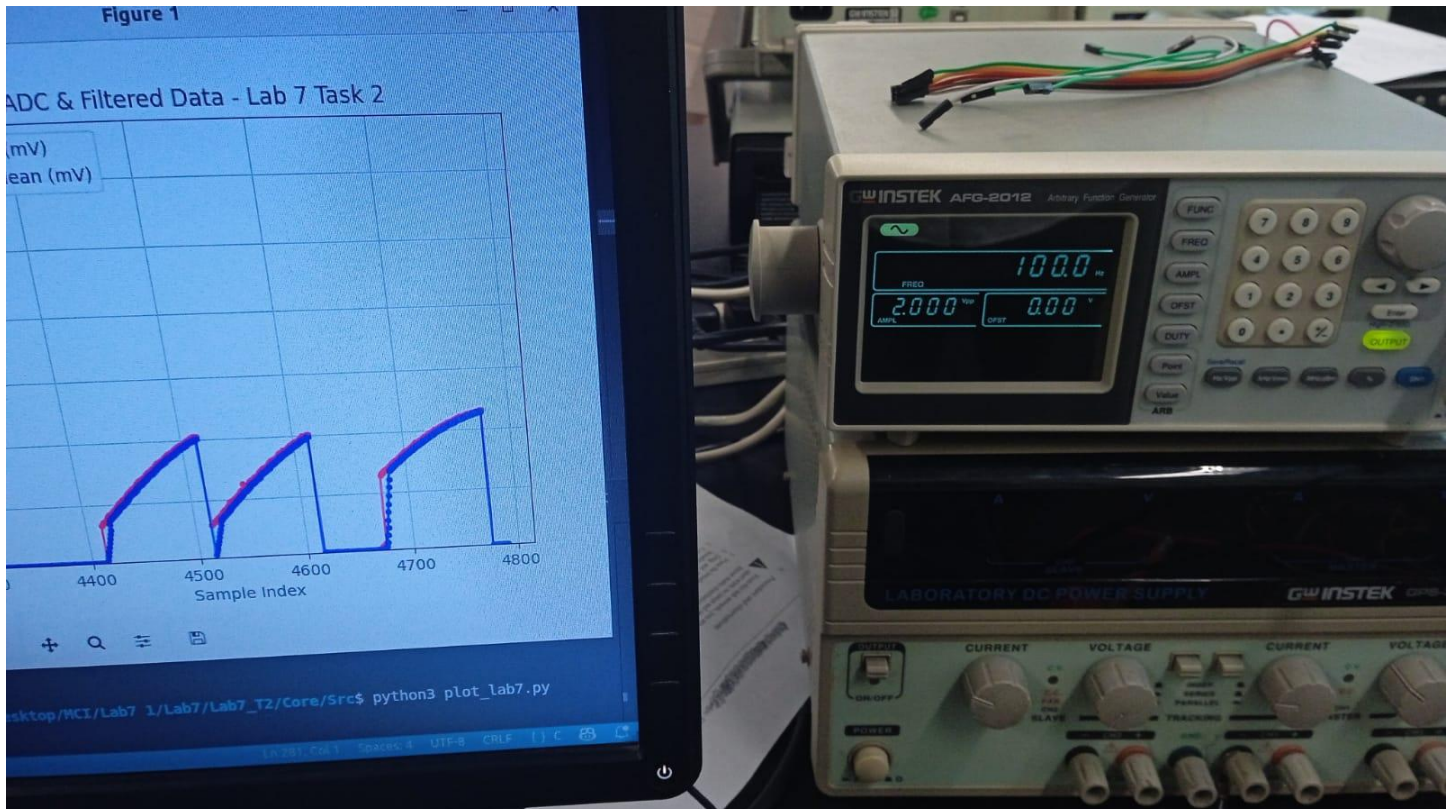
10 HZ



50 HZ:



100 HZ:



EVALUATION QUESTION:

a) Explain the relationship between ADC resolution and quantization error. Why does increasing resolution reduce error

The resolution of an ADC determines the number of discrete levels used to represent an analog signal, given by 2^N for an N-bit ADC. The input voltage range is divided into these levels, and the smallest detectable change in input, known as the least significant bit (LSB), is equal to $V_{ref}/2^N$. Quantization error arises because the ADC approximates the continuous analog signal to the nearest discrete level, with a maximum error of $\pm\text{LSB}/2$. As the resolution increases, the number of levels increases and the LSB size decreases, which directly reduces the quantization error. Therefore, a higher-resolution ADC provides a more accurate digital representation of the analog input signal.